



Hospital

Faites adhérer les parties prenantes avec un POC

Architecte Logiciel P11

Emmy Lemonnier

openclassrooms

Mise à jour des documents d'architecture suite au Proof of Concept

Table des matières

1. Objet du document	3
2. Mises à jour proposées pour la Déclaration des travaux d'architecture	3
Mise en œuvre de la PoC	3
Front-end Vue.js.....	3
Respect des principes	4
2.2 Livrables de travail.....	4
2.3 Critères et procédures d'acceptation	4
Critères retenus pour juger la réussite de la PoC :	4
Procédures d'acceptation :.....	5
3. Mises à jour proposées pour le Document de définition de l'architecture	5
3.1 Architecture métier, données, applications, technologie	5
Présentation du prototype microservices.....	5
3.2 Analyse de l'écart et prochaines étapes.....	6
Prochaines étapes :	7
Conclusion et utilité de ce document :	8

1. Objet du document

Le présent document vise à **compléter et enrichir** :

- **La Déclaration des travaux d'architecture**, qui formalise le cadre, les livrables et les critères d'acceptation de la preuve de concept (PoC),
- **Le Document de définition de l'architecture**, qui décrit le contexte métier, l'architecture cible, ainsi que l'analyse de l'écart entre l'existant et le futur système.

La PoC, réalisée en **Java Spring Boot** (trois microservices distincts) avec un **front-end Vue.js**, a pour but de valider la faisabilité d'une architecture microservices pour la gestion des hôpitaux, de l'authentification et d'un service de calcul de distance.

2. Mises à jour proposées pour la Déclaration des travaux d'architecture

2.1 Approche architecturale et Processus d'architecture

Dans la section *Approche architecturale* (ou *Processus d'architecture*), on peut ajouter :

Mise en œuvre de la PoC

Dans le cadre de la preuve de concept, nous avons implémenté :

- **Un microservice “User/Auth”** pour gérer l'authentification et les utilisateurs (Java Spring Boot + JWT).
- **Un microservice “Hospital”** pour gérer les informations d'un hôpital (nom, localisation, spécialités éventuelles, etc.) et permettre d'ajouter un hôpital via une API REST.
- **Un microservice “Distance”** pour calculer ou simuler la distance entre deux emplacements, appelé directement par le microservice Hospital.

Front-end Vue.js

- Un front minimaliste développé en Vue.js permet de présenter l'interface utilisateur :
 - Affichage de la liste d'hôpitaux,
 - Formulaire d'ajout d'hôpital,

- Gestion d'un login/logout basique pour tester la partie auth.

Respect des principes

- Nous avons appliqué une **séparation des responsabilités** (principe B2), en cloisonnant l'authentification d'une part, la gestion d'un hôpital d'autre part.
- L'intégration et la livraison se font de manière continue (principe B3), avec un pipeline CI/CD (même rudimentaire) lancé à chaque push.
- Les tests sont automatisés (principe B4), incluant des tests unitaires et quelques tests d'intégration pour vérifier la communication entre Hospital et Distance.

Ces développements ont été réalisés **en mode prototype**, dans un environnement isolé et **sans données réelles** de patients (afin de respecter les contraintes RGPD et la sécurité).

2.2 Livrables de travail

Dans la section *Livrables de travail*, on peut ajouter :

- **Code source** des trois microservices Java Spring Boot + la configuration du front-end Vue.js, hébergés sur un dépôt Git.
- **Pipelines CI/CD** : exécution de builds et de tests automatisés pour chaque service.
- **Rapport de tests** : un résumé des tests unitaires et d'intégration effectués.
- **Présentation PowerPoint** détaillant l'architecture, le fonctionnement de la PoC et les futures recommandations.

L'objectif étant de démontrer la faisabilité, ces livrables attestent d'une première intégration fonctionnelle.

2.3 Critères et procédures d'acceptation

Dans la section *Critères et procédures d'acceptation*, on peut préciser :

Critères retenus pour juger la réussite de la PoC :

1. Vérifier que la **communication entre microservices** (Hospital ↔ Distance) fonctionne en REST, avec un temps de réponse stable sur un petit volume de requêtes.
2. **Authentification opérationnelle** : seul un utilisateur authentifié peut ajouter un hôpital.
3. Mise en place d'une **intégration continue** : tout commit déclenche la compilation et les tests unitaires.

4. **Front Vue.js** consommant l'API de manière fluide, illustrant la faisabilité d'une interface utilisateur.

Procédures d'acceptation :

- Revue de code et revue d'architecture (hospital, distance, auth) par le comité d'architecture.
 - Tests exécutés et validés sur la branche principale.
 - Démonstration d'usage concret : connexion, ajout d'un hôpital et appel du microservice Distance depuis le front.
-

3. Mises à jour proposées pour le Document de définition de l'architecture

3.1 Architecture métier, données, applications, technologie

Dans la partie décrivant l'architecture ou le "Modèle de domaine" (architecture métier, système d'information, etc.), on peut ajouter :

Présentation du prototype microservices

- **Hospital Service** : gère la ressource "Hôpital" (ID, nom, adresse, spécialités, sous spécialité). L'API permet de lister, consulter et ajouter des hôpitaux. À terme, ce service pourrait inclure la gestion de la disponibilité des lits.
- **Distance Service** : appelé par Hospital pour récupérer la distance (fictive ou réelle) entre deux points. Ce service se substitue à un calcul plus poussé qui, dans la version finale, pourrait inclure temps de trajet, contraintes géographiques, et la géo-localisation
- **User/Auth Service** : endosse la fonction "authentification" et "autorisation" pour tous les autres microservices, évitant ainsi tout couplage direct dans Hospital ou Distance.

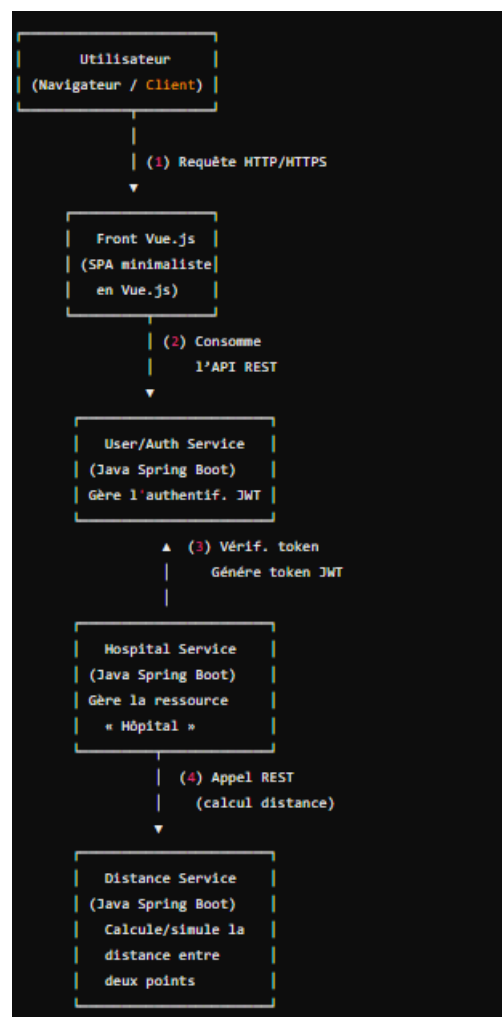
Technologies mises en œuvre

- **Java Spring Boot** : pour la réalisation de chaque microservice. Les endpoints sont exposés en REST (contrôleurs @RestController).
- **Vue.js** : front-end SPA (Single Page Application) interrogeant les endpoints de l'Auth, Hospital, etc.

- **Mécanismes de sécurité** : JWT (JSON Web Token) pour l'authentification ; un utilisateur doit s'authentifier pour déclencher certaines opérations (ex. ajout d'hôpital).

Respect des principes

- Principes B2 (séparation des préoccupations) et B3 (intégration continue) ont été respectés, validant la mise en place du pipeline.
- Principe B4 (tests automatisés) : la PoC inclut des tests unitaires pour chaque microservice, ainsi que quelques tests d'intégration pour vérifier la communication interservices.



3.2 Analyse de l'écart et prochaines étapes

Dans la section *Analyse de l'écart* ou *Roadmap*, on peut ajouter :

Constats suite à la PoC :

1. **Hospital Service** : les endpoints CRUD de base fonctionnent. Cependant, la gestion de la “disponibilité des lits” n’est pas encore implémentée, et le modèle de données est volontairement simplifié.
2. **Distance Service** : il s’agit d’une implémentation minimaliste (mock ou calcul simpliste). Pour l’architecture cible, il faudra soit intégrer un service de cartographie, soit brancher une API tierce pour des calculs plus fiables.
3. **Authentification** : la PoC démontre la faisabilité d’un microservice Auth. Restent à explorer la gestion fine des rôles, la sécurisation avancée, la tolérance aux pannes, etc.
4. **Front-end Vue.js** : valide la communication asynchrone avec un design modulaire. Son périmètre reste minimal, mais suffisant pour illustrer les appels aux microservices.

Prochaines étapes :

- Créer la couche “event-driven” : intégrer un bus d’événements (Kafka, RabbitMQ) pour diffuser les changements d’état (ex. ajout d’un hôpital, changement de disponibilité de lits).
- Gérer des données patient anonymisées : afin de tester l’intégration future de la logique d’attribution de lits, tout en restant conforme à la protection des données.
- Autres micro-services : gestions des lits, prises en compte des symptômes
- Compléter la gestion des hôpitaux : CRUD complet.
- Mettre en place une vraie géo-localisation pour le calcul des distances.
- Observabilité et résilience : outiller l’infrastructure (monitoring, logs centralisés, metrics) et prévoir des mécanismes de fallback en cas de panne (principe B1).
- Sécurité renforcée : finaliser la gestion fine des rôles et autorisations, mettre en place des tests de sécurité (principe B5).

Cette PoC confirme la pertinence d’une architecture microservices et valide certaines options techniques. Elle n’inclut pas encore toutes l’architectures complètes, qui feront l’objet d’itérations ultérieures.

Conclusion et utilité de ce document :

Le présent document peut être remis en complément aux PDF existants ou intégré dans un nouveau livrable Word pour consolider toute la documentation. Il contient l'essentiel de ce qui a été réalisé dans le cadre du PoC et ce qui devrait apparaître dans les parties pertinentes des deux documents d'architecture.